

Руководство разработчика

Med Doctor Assignment Service. Версия документации: 2026-04-29.

Содержание

Руководство разработчика

1. Назначение документа
2. Технологический профиль
3. Архитектурная схема кода
4. Ключевые классы
5. Алгоритм назначения
6. Особенность очереди «Врач не назначен»
7. Локальный запуск
 - 7.1. Запуск тестов
 - 7.2. Запуск приложения
 - 7.3. Безопасный локальный профиль
8. Конфигурационные инварианты
9. Точки расширения
10. Правила работы с Orchestra REST
11. Поведение при ошибках
12. Тестовая стратегия
13. Чек-лист перед merge
14. Быстрые ссылки внутри проекта

1. Назначение документа

Документ описывает архитектуру, точки расширения, локальный запуск, тестирование и правила безопасной доработки **Med Doctor Assignment Service**. Сервис автоматически подбирает следующую услугу визита из очереди «Врач не назначен» и переводит визит в очередь, подходящую врачу или выбранную med-robot.

2. Технологический профиль

Область	Используется
Язык	Java 8
Framework	Micronaut 3.5.2
Сборка	Maven Wrapper
HTTP-клиент	Micronaut HTTP Client + Reactor
Websocket/STOMP	Spring WebSocket/Messaging + SockJS endpoint Orchestra
Логирование	Logback
Тесты	JUnit 5, Micronaut Test
Основная интеграция	Orchestra 6 REST + события <code>/qpevents/events</code>
Опциональная интеграция	med-robot REST API

3. Архитектурная схема кода

Пакет	Ответственность	Что нельзя смешивать
<code>adapter.orchestra</code>	REST-клиент Orchestra, cookie/session handling, reactive response handling	доменный выбор услуги и очереди.
<code>adapter.medrobot</code>	REST-клиент med-robot и DTO ответа	fallback-решения и проверка маршрута визита.
<code>adapter.gateway</code>	транспортные gateway-реализации для домена	правила выбора услуги.
<code>branchgetter</code> , <code>orchestracache</code>	получение и хранение справочников отделений	mutation workflow визита.
<code>event</code> , <code>websocket</code>	подписка на события, корреляция посадки врача	REST-пути Orchestra.
<code>domain.model</code>	доменные модели визита/услуг	HTTP DTO, заголовки, cookies.
<code>domain.service</code>	алгоритм назначения и выбора	низкоуровневый HTTP-код.
<code>schedule</code>	polling reconciliation	бизнес-логика выбора.
<code>util</code>	branch lock, дедупликация событий/визитов	интеграционные контракты.
<code>diagnostics</code>	логирование эффективной конфигурации	изменение состояния визита.

Главное правило доработки: доменный слой не должен знать, по какому URL выполняется `assign-service` или как хранится cookie Orchestra. Он должен работать через `VisitWorkflowGateway`, `OrchestraMetadataGateway`, `ServicePointContextGateway`, `MedRobotOptimalServiceGateway`.

4. Ключевые классы

Класс	Назначение
<code>AutonomousMedicalExamAssignmentService</code>	основной цикл назначения визитов врачу.
<code>MedRobotAwareDoctorServiceSelectionService</code>	выбор услуги через med-robot с fallback к локальному алгоритму.
<code>DefaultDoctorServiceMatcher</code>	локальное сопоставление услуг визита и услуг/очередей врача.
<code>DefaultLoggedDoctorContextResolver</code>	восстановление контекста врача по branch/service point/session/work profile.
<code>DefaultVisitAssignmentExecutor</code>	выполнение assign/transfer с защитными проверками.
<code>DoctorAssignmentEventHandler</code>	обработка событий Orchestra и запуск assignment cycle.
<code>UserSessionReadinessCoordinator</code>	корреляция <code>USER_SERVICE_POINT_SESSION_START</code> -> <code>SET_WORK_PROFILE</code> .
<code>WorkProfileExpansionTriggerEvaluator</code>	повторный запуск при расширении рабочего профиля врача.
<code>PollingReconciliationJob</code>	страхующий запуск по расписанию.
<code>OrchestraSessionCookieStore</code>	разделение cookie для GET и mutating REST-вызовов.
<code>RuntimeConfigurationLogger</code>	логирование эффективных настроек при старте.

5. Алгоритм назначения

1. Получить `DoctorContext` : branch, service point, staff, work profile.
2. Проверить, разрешен ли branch через `allowed-branches`.
3. Получить доступные врачу очереди и услуги из `branch cache`.
4. Прочитать визиты очереди «Врач не назначен».
5. Для каждого визита прочитать детали маршрута.
6. Построить `route fingerprint`, чтобы не обрабатывать один и тот же маршрутный шаг повторно.
7. Выбрать услугу:
 - локально через пересечение маршрута визита и услуг врача;
 - или через med-robot, если `application.med-robot.enabled=true`.
1. Перед мутацией перечитать визит, если `recheck-visit-before-transfer=true`.
2. Выполнить `assign-service`, если это требуется состоянием визита.
3. Выполнить `transfer-visit` в целевую очередь.
4. Записать результат и диагностические признаки в лог.

6. Особенность очереди «Врач не назначен»

В реальных логах встречается ситуация: визит уже находится в очереди «Врач не назначен», и `currentService.serviceId` также соответствует услуге «Врач не в системе» или аналогичной служебной услуге. При этом `unservedVisitServices` может быть пустым. Для этого сценария поддержан режим `med-robot` по номеру талона:

```
application:
  med-robot:
    enabled: true
    request-body-mode: TICKET_NUMBER_PLAIN_TEXT
```

Разработчику важно не завязывать вызов `med-robot` только на наличие непустого `unservedVisitServices`. Если у визита есть `ticketId` и текущая услуга, сервис может запросить `med-robot` по номеру талона.

7. Локальный запуск

7.1. Запуск тестов

```
./mvnw clean test
```

Windows:

```
.\mvnw.cmd clean test
```

7.2. Запуск приложения

```
./mvnw mn:run -Dmicronaut.config.files=./src/main/resources/application.yml
```

или из `jar`:

```
./mvnw clean package
java -jar target/med-doctor-assignment-service-1.0.0-SNAPSHOT.jar --
micronaut.config.files=./config/application-test.yml
```

7.3. Безопасный локальный профиль

Для локальной отладки без реальных изменений используйте:

```
application:
  assignment:
    dry-run: true
    max-visits-per-cycle: 3
    allowed-branches: [1]
  websocket:
    enabled: false
  med-robot:
    enabled: false
```

8. Конфигурационные инварианты

Инвариант	Почему важен
<code>dry-run=true</code> на первом запуске	исключает случайные мутации в Orchestra.
<code>allowed-branches</code> не пустой на стенде	ограничивает область воздействия.
<code>abort-cycle-on-forbidden-mutation=true</code>	предотвращает шквал одинаковых 403/500.
<code>recheck-visit-before-transfer=true</code>	защищает от гонок между несколькими операторами/событиями.
<code>service-point-open-trigger-enabled=false</code>	<code>SERVICE_POINT_OPEN</code> может приходить раньше готовности профиля.
<code>user-service-point-session-start-trigger-enabled=true</code>	основной безопасный trigger через корреляцию с <code>SET_WORK_PROFILE</code> .
<code>send-cookies-in-handshake=false</code>	REST-cookie и websocket handshake не смешиваются.
<code>replay-mutation-cookies=false</code>	защищает от повторных 403 после mutating-cookie.

9. Точки расширения

Нужно изменить	Расширять здесь	Проверить тестами
новый алгоритм локального выбора услуги	DoctorServiceMatcher	DoctorServiceMatcherTest.
новые правила доступных врачу услуг	DoctorAvailableServicesResolver	тест с work profile/queue/service mapping.
новый REST-контракт Orchestra	ConfigurableVisitWorkflowGateway + AssignmentProperties.ExperimentalEndpoints	тест gateway и runbook scenario.
иной контракт med-robot	MedRobotRestClient, MedRobotOptimalServiceGatewayImpl	MedRobotRestClientContractTest.
новый trigger события	DoctorAssignmentEventHandler	DoctorAssignmentEventHandlerTriggerTest.
другая логика сессии врача	UserSessionReadinessCoordinator, DefaultLoggedDoctorContextResolver	тесты корреляции и контекста.
дополнительная диагностика конфигурации	RuntimeConfigurationLogger	RuntimeConfigurationLoggerTest.

10. Правила работы с Orchestra REST

1. Все REST-вызовы должны передавать Basic Auth.
2. Cookie, полученные из GET-ответов, используются только для следующих GET-запросов.
3. Cookie, полученные из PUT/POST-ответов, не должны автоматически попадать в GET и websocket handshake.
4. Для mutating-запросов replay cookie по умолчанию отключен.
5. Endpoint-ы workflow должны оставаться конфигурируемыми в application.assignment.experimental-endpoints.
6. Нельзя считать HTTP 200 достаточным признаком успеха assign: нужно анализировать тело ответа и userState.

11. Поведение при ошибках

Ошибка	Правильная реакция кода
Orchestra недоступна на старте	не завершать JVM, запланировать повторный bootstrap/refresh.
refresh branch cache упал	сохранить старый кэш, записать ошибку, повторить позже.
websocket потерян	переподключаться с задержкой, polling должен оставаться страховкой.
med-robot недоступен	FALLBACK_TO_LOCAL или SKIP_VISIT согласно настройке.
assign вернул INACTIVE	считать контекстной ошибкой, не делать transfer в том же цикле.
assign вернул NO_STARTED_SERVICE_POINT_SESSION	остановить текущий цикл и дождаться стабилизации сессии.
transfer получил 403/500	записать request/response context, остановить цикл при abort-cycle-on-forbidden-mutation=true.

12. Тестовая стратегия

Минимальный набор тестов при изменении алгоритма:

```
./mvnw test -Dtest=AutonomousMedicalExamAssignmentServiceTest
./mvnw test -Dtest=MedRobotAwareDoctorServiceSelectionServiceTest
./mvnw test -Dtest=DoctorAssignmentEventHandlerTriggerTest
./mvnw test -Dtest=SetWorkProfileIntegrationTest
./mvnw test -Dtest=PollingReconciliationJobTest
./mvnw test -Dtest=DocumentationAssetsTest
```

Перед передачей сборки на стенд желательно выполнять полный прогон:

```
./mvnw clean test
```

13. Чек-лист перед merge

- [] Код компилируется на Java 8.
- [] Полный `./mvnw clean test` проходит локально.
- [] Новая логика покрыта unit/integration тестами.
- [] Не добавлены жестко зашитые URL Orchestra вне конфигурации.
- [] Не смешаны доменные решения и HTTP details.
- [] Логи содержат branch id, visit id, staff id и trigger source для диагностики.
- [] Обновлено README/RUNBOOK/CONFIGURATION_REFERENCE при изменении поведения.
- [] Для опасных изменений описан безопасный `dry-run` сценарий.

14. Быстрые ссылки внутри проекта

- Основная документация: `README.md` .
- Конфигурация: `CONFIGURATION_REFERENCE.md` .
- Тестовый стенд: `docs/TEST_STAND_SETUP.md` .
- Режимы работы: `OPERATION_MODES.md` .
- REST-контракты Orchestra: `ORCHESTRA_REST_CONTRACTS.md` .
- Интеграция med-robot: `MED_ROBOT_INTEGRATION.md` .
- Runbook поддержки: `RUNBOOK.md` .